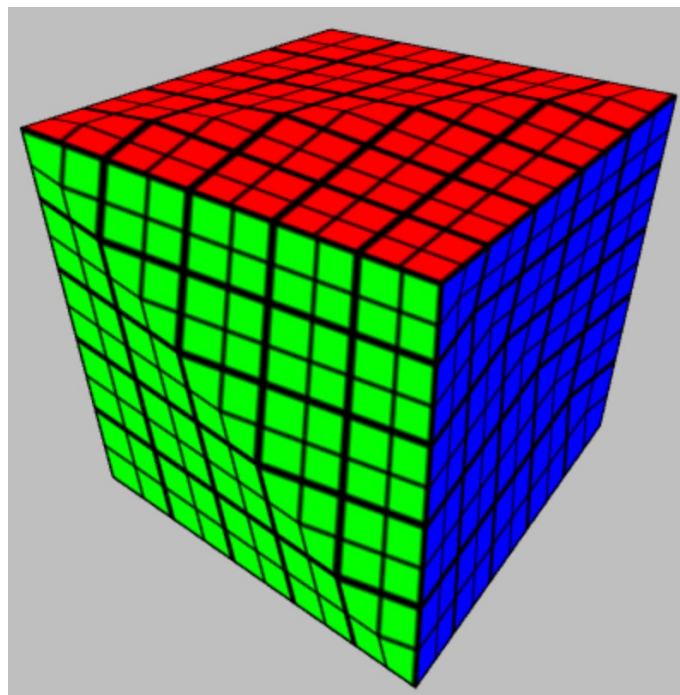




Zbuffer e Proiezione Prospettica con Profondità





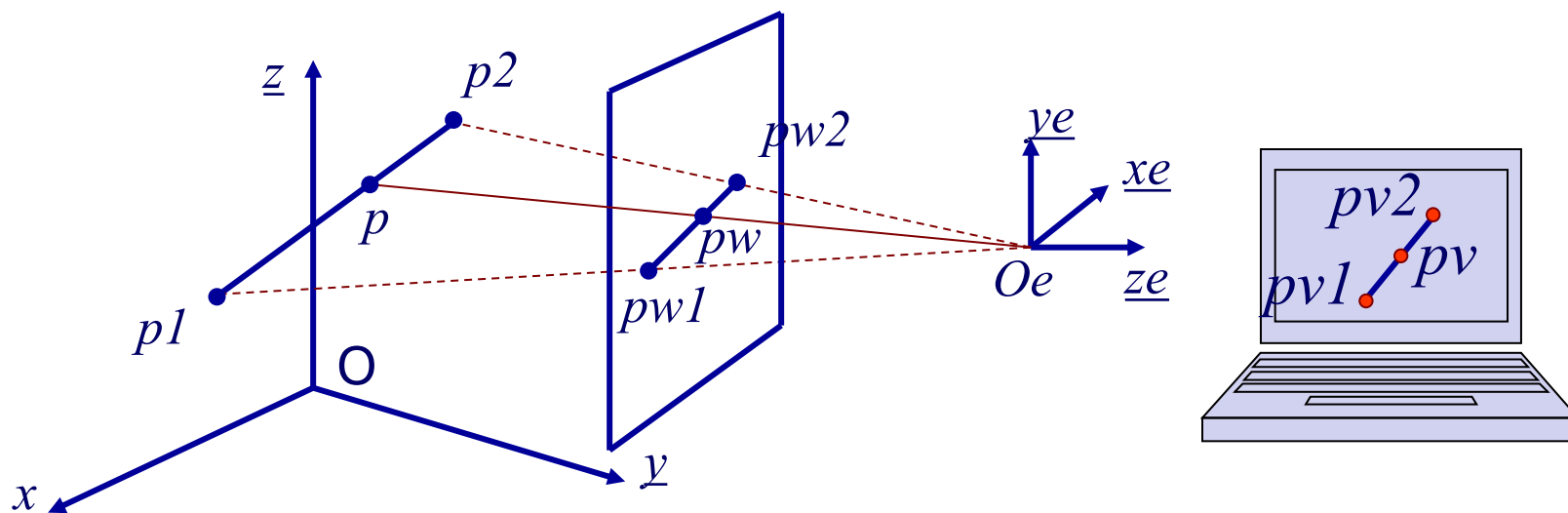
Algoritmo Z-buffer

Riprendiamo l'algoritmo Z-Buffer (o depth buffer) per la rimozione di parti nascoste, ...

```
void zbuffer() {  
  
    for (y = 0; y < Vymax; y++)  
        for(x = 0; x < Vxmax; x++) {  
            FrameBuffer(x,y) = BACK_Col;  
            ZBuffer(x,y,INF_Val);  
        }  
  
    for (every triangle)  
        for (all pixel (px,py) of the projected triangle) {  
  
            pZ = computeZ(px,py);  
            if (pZ < ZBuffer(px,py)) //pixel is visible  
            {  
                FrameBuffer(px,py) = computeColor(px,py);  
                ZBuffer(px,py) = pZ;  
            }  
        }  
}
```

Profondità di un Pixel

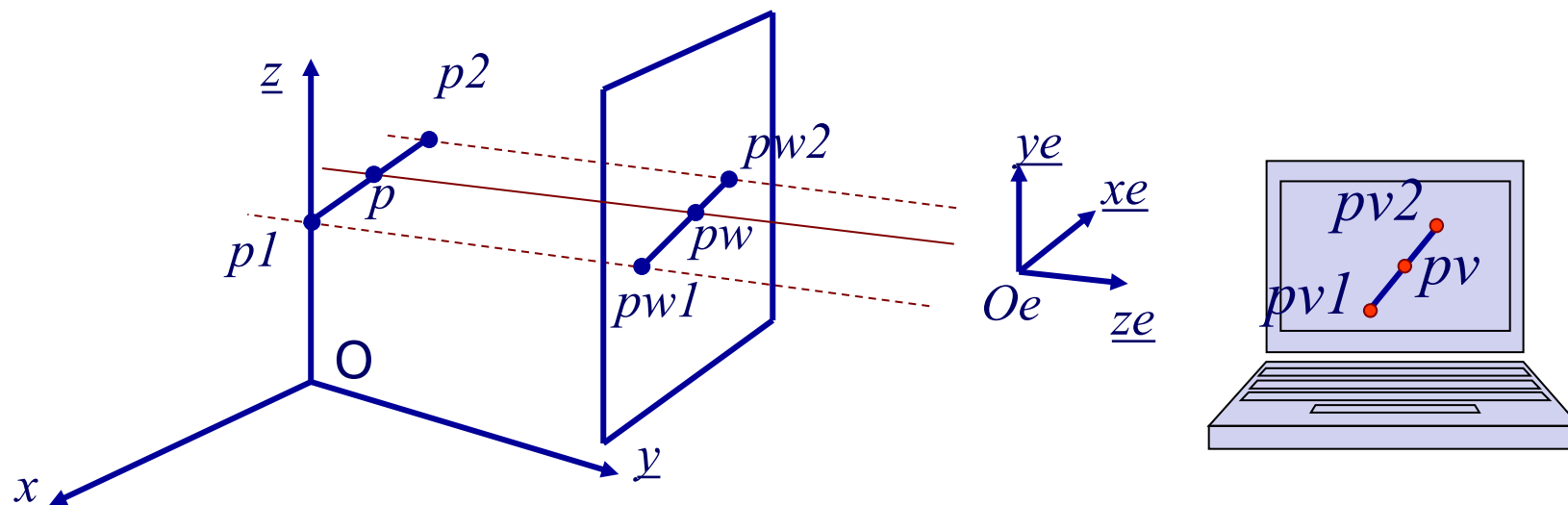
... vogliamo affrontare il problema di come calcolare la profondità di un pixel.



Problema: siano $pw1$ e $pw2$ i proiettati di $p1$ e $p2$ e $pv1$, $pv2$ i trasformati window-viewport di $pw1$ e $pw2$; se pv è un punto/pixel del segmento $pv1$ - $pv2$, si vuole determinare la 'sua' profondità, cioè la coordinata z di p nel sistema $(\underline{x}_e, \underline{y}_e, \underline{z}_e, Oe)$ dell'osservatore di cui pw è la proiezione.

Profondità di un Pixel

Se la proiezione è parallela ortogonale ...



.... scriviamo il segmento di estremi $pv1$ e $pv2$ in forma baricentrica/parametrica:

$$pv(\tau) = (1 - \tau) pv1 + \tau pv2 \quad \text{con } \tau \in [0, 1].$$

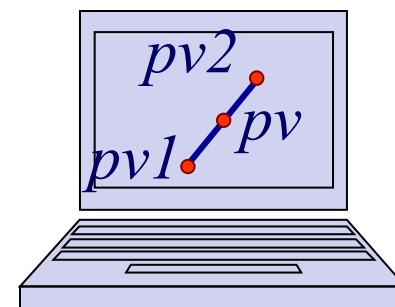
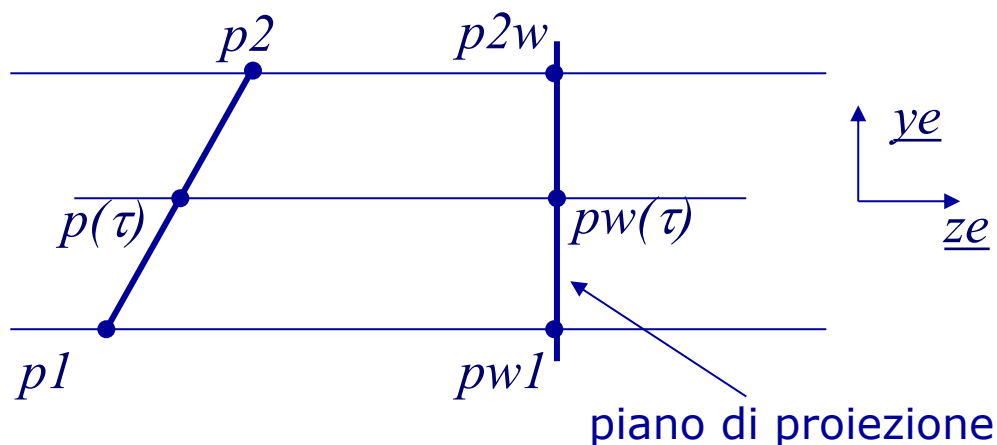
Per ogni punto pv del segmento determiniamo la sua coordinata τ e gli associamo come profondità (coordinata z) quella del punto p del segmento di estremi $p1$ e $p2$, ottenuto come:

$$p(\tau) = (1 - \tau) p1 + \tau p2$$

Profondità di un Pixel

... ed il procedimento sarebbe CORRETTO, infatti, la proiezione parallela ortogonale è una trasformazione lineare, così come la trasformazione window-viewport.

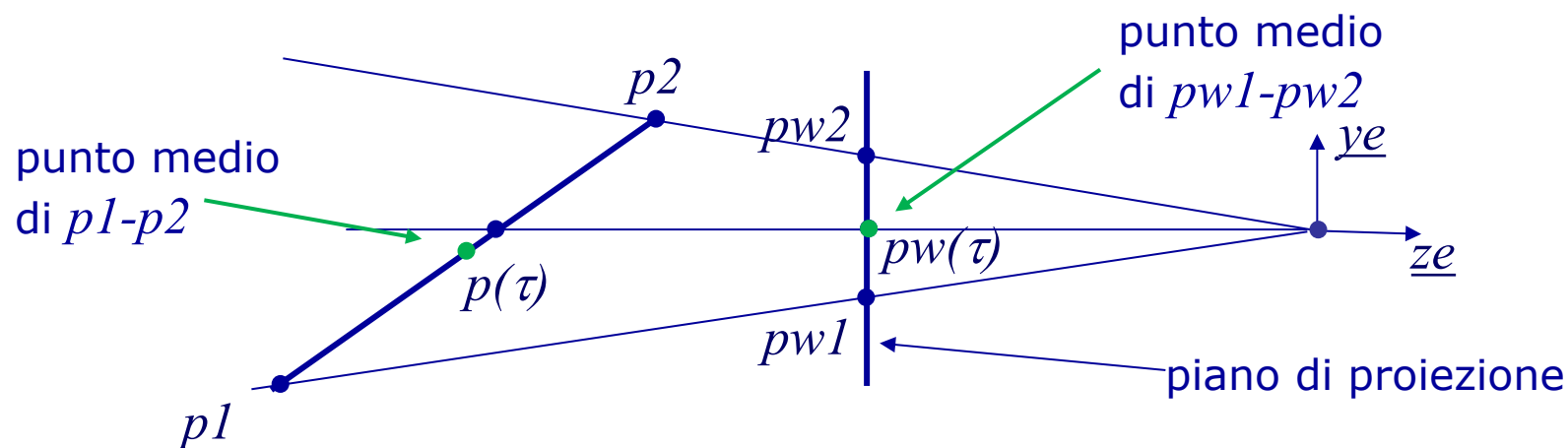
Caso di proiezione parallela ortogonale (vista 2D)



Profondità di un Pixel

Ma nel caso di proiezione prospettica, questo procedimento NON è CORRETTO, infatti la proiezione prospettica non è una trasformazione lineare.

Caso di proiezione prospettica (vista 2D)

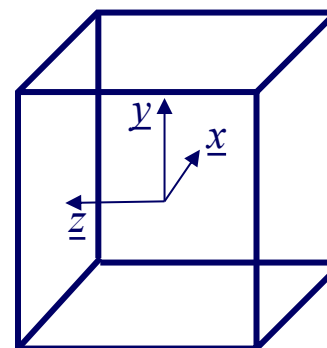
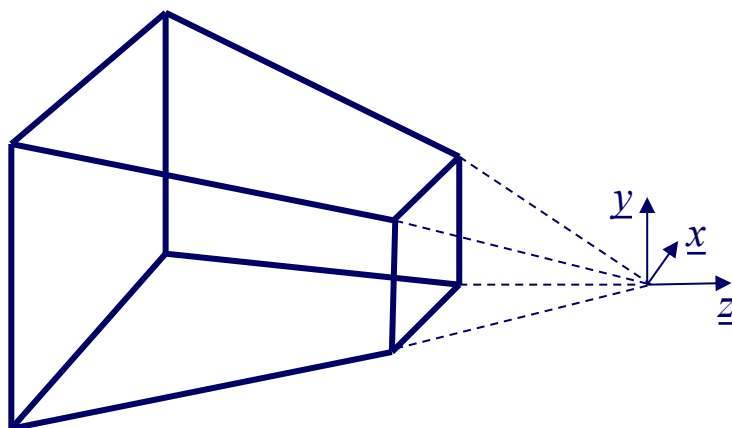


Nella figura si può osservare che il punto $p(\tau)$ non è il punto che proiettato porta a $pw(\tau)$.

Prospettiva con Profondità

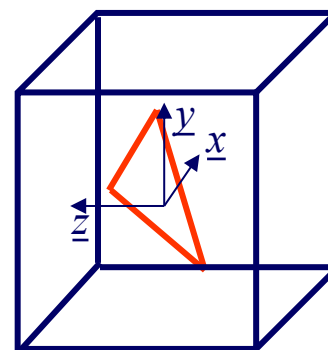
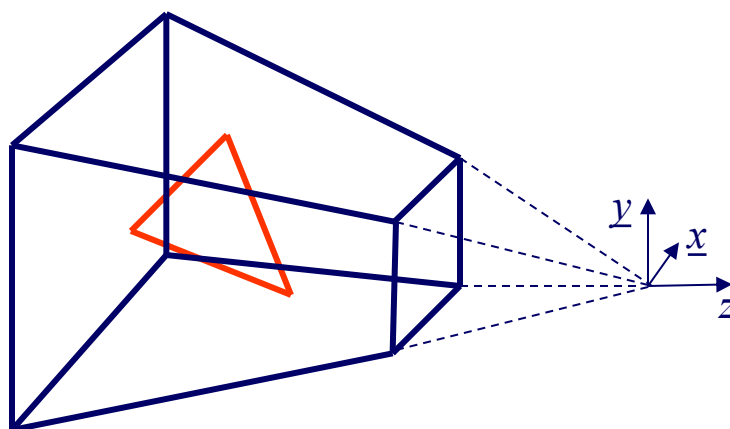
L'obiettivo è quello di definire una trasformazione prospettica dal "sistema 3D dell'osservatore" al "sistema 3D del piano di proiezione" con lo scopo di deformare lo spazio 3D di un tronco di piramide in uno spazio 3D di un cubo (cubo WebGL).

Questa deformazione permetterà di interpretare la **proiezione prospettica** come una **proiezione parallela** e poter applicare le coordinate baricentriche per determinare la profondità dei pixel.



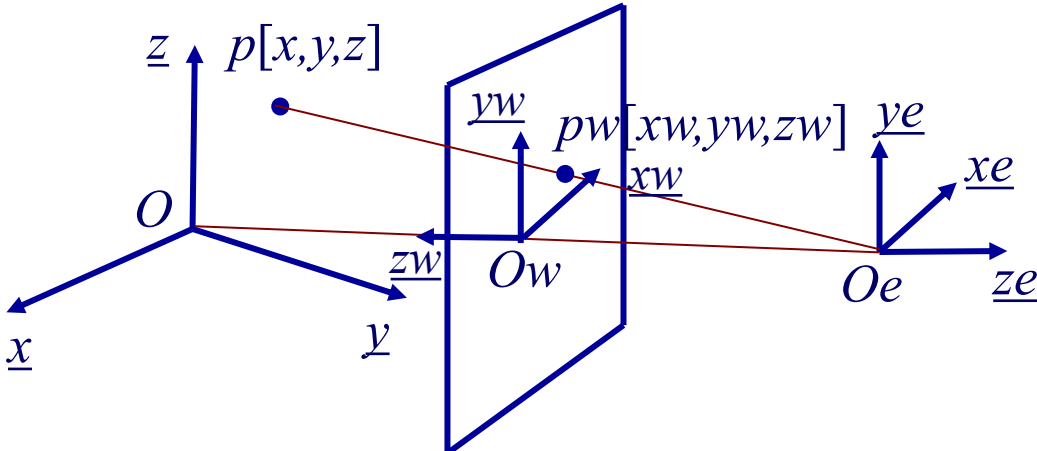
Prospettiva con Profondità

La trasformazione che si cerca deve essere tale da mantenere l'ordine di profondità dei punti trasformati, nella stessa relazione dei punti originali ed ancora deve essere tale da trasformare rette in rette e piani in piani in quanto una geometria poligonale piana sottoposta a questa trasformazione dovrà rimanere poligonale piana.



Prospettiva con Profondità

Teorema. La seguente proiezione

$$\left\{ \begin{array}{l} xw = -x/z \\ yw = -y/z \\ zw = -(\alpha + \beta/z) \end{array} \right. \quad (1)$$


comporta una trasformazione dal "sistema 3D dell'osservatore" ($\underline{x_e}, \underline{y_e}, \underline{z_e}, O_e$) al "sistema 3D del piano di proiezione" ($\underline{x_w}, \underline{y_w}, \underline{z_w}, O_w$) con la proprietà di trasformare rette in rette e piani in piani, per $\alpha, \beta \neq 0$



Prospettiva con Profondità

Verifichiamo per rette: se un segmento viene trasformato in un segmento dovrà esistere una relazione tra i parametri delle due forme parametriche.

Sia $p(t) = (1-t)p_1 + t p_2 \quad t \in [0, 1]$

un segmento nello spazio dell'osservatore con $p_i = [x_i, y_i, z_i] \quad i=1,2$.

Siano $p w_i = [x w_i, y w_i, z w_i] \quad i=1,2$ i punti ottenuti applicando la proiezione, cioè $p w_i = [-x_i/z_i, -y_i/z_i, -(\alpha + \beta/z_i)]$ ed il segmento relativo sia $p w(\tau) = (1-\tau) p w_1 + \tau p w_2 \quad \tau \in [0, 1]$.

Che relazione c'è tra t e τ ? La relazione si ricaverà imponendo che $p(t)$ venga trasformato in $p w(\tau)$; guardiamo l'ascissa del punto trasformato e a cosa corrisponde:

$$-\frac{(1-t)x_1 + t x_2}{(1-t)z_1 + t z_2} = -(1-\tau)\frac{x_1}{z_1} - \tau \frac{x_2}{z_2}$$



Prospettiva con Profondità

i passaggi:

$$\frac{(1-t)x_1 + tx_2}{(1-t)z_1 + tz_2} = (1-\tau)\frac{x_1}{z_1} + \tau\frac{x_2}{z_2}$$

$$\frac{(1-t)x_1 + tx_2}{(1-t)z_1 + tz_2} = \frac{x_1}{z_1} + \tau\left(\frac{x_2}{z_2} - \frac{x_1}{z_1}\right)$$

ricaviamo τ :

$$\tau = \left(\frac{(1-t)x_1 + tx_2}{(1-t)z_1 + tz_2} - \frac{x_1}{z_1} \right) \frac{1}{\left(\frac{x_2}{z_2} - \frac{x_1}{z_1} \right)}$$

e facendo un po' di conti:

$$\tau = \frac{tz_2}{(1-t)z_1 + tz_2} \quad (2)$$

Prospettiva con Profondità

Dalla relazione (2), si può ricavare banalmente la (3):

$$(2) \quad \tau = \frac{t \cdot z_2}{(1-t) \cdot z_1 + t \cdot z_2} \quad (1-t) = \frac{(1-t) \cdot z_1}{(1-t) \cdot z_1 + t \cdot z_2} \quad (3)$$

e ricavando t , si ottengono le relazioni inverse (4) e (5):

$$(4) \quad t = \frac{\tau \cdot z_1}{\tau \cdot z_1 + (1-\tau) \cdot z_2} \quad (1-t) = \frac{(1-\tau) \cdot z_2}{\tau \cdot z_1 + (1-\tau) \cdot z_2} \quad (5)$$

Prospettiva con Profondità

Verifichiamo che la proiezione (1) trasforma rette in rette. Si considera la proiezione di un punto $p(t)$ della retta per $p1$ e $p2$ e si verifica che stia sulla retta per $pw1$ e $pw2$.

$$pw = \left[-\frac{(1-t) \cdot x_1 + tx_2}{(1-t) \cdot z_1 + t \cdot z_2}, -\frac{(1-t) \cdot y_1 + ty_2}{(1-t) \cdot z_1 + t \cdot z_2}, -\left(\alpha + \frac{\beta}{(1-t) \cdot z_1 + t \cdot z_2}\right) \right]$$

Consideriamo la prima coordinata di pw e moltiplichiamo e dividiamo per z_1 e z_2 :

$$-\frac{(1-t) \cdot x_1}{(1-t) \cdot z_1 + t \cdot z_2} \frac{z_1}{z_1} - \frac{tx_2}{(1-t) \cdot z_1 + t \cdot z_2} \frac{z_2}{z_2}$$

$$(2) \quad \tau = \frac{t \cdot z_2}{(1-t) \cdot z_1 + t \cdot z_2}$$

$$(1-\tau) = \frac{(1-t) \cdot z_1}{(1-t) \cdot z_1 + t \cdot z_2} \quad (3)$$



Prospettiva con Profondità

Utilizzando le relazioni (2) e (3) si ha:

$$-(1 - \tau) \frac{x_1}{z_1} - \tau \frac{x_2}{z_2}$$

In modo analogo per la seconda coordinata di pw .

Consideriamo ora la terza coordinata di pw ; dalle (2) e (3), dividendo per z_2 la prima e z_1 la seconda e sommando si ottiene:

$$\frac{\tau}{z_2} + \frac{(1 - \tau)}{z_1} = \frac{1}{(1 - t) \cdot z_1 + t \cdot z_2}$$

vediamo come sostituirla nella terza coordinata di pw :



Prospettiva con Profondità

$$\begin{aligned} -\left(\alpha + \frac{\beta}{(1-t) \cdot z_1 + t \cdot z_2}\right) &= -\alpha(1-\tau) - \alpha\tau - \beta\left(\frac{1-\tau}{z_1} + \frac{\tau}{z_2}\right) = \\ &= -(1-\tau)\left(\alpha + \frac{\beta}{z_1}\right) - \tau\left(\alpha + \frac{\beta}{z_2}\right) \end{aligned}$$

e quindi le coordinate di pw si possono riscrivere:

$$pw = \left[-(1-\tau)\frac{x_1}{z_1} - \tau\frac{x_2}{z_2}, -(1-\tau)\frac{y_1}{z_1} - \tau\frac{y_2}{z_2}, -(1-\tau)\left(\alpha + \frac{\beta}{z_1}\right) - \tau\left(\alpha + \frac{\beta}{z_2}\right) \right]$$

questa è la verifica che si tratta di un punto sul segmento $pw1$ $pw2$ relativo al parametro τ corrispondente di t e quindi che rette vengono trasformate in rette.



Prospettiva con Profondità

In conclusione le relazioni (2) e (3) forniscono i parametri τ e $(1 - \tau)$ che se combinati con le coordinate dei vertici proiettati permettono di ottenere le coordinate proiettate dei punti originali, in particolare per la componente z :

$$\begin{aligned}zw &= -(1 - \tau)(\alpha + \beta / z_1) - \tau(\alpha + \beta / z_2) \\&= -\frac{(1 - t)z_1}{(1 - t)z_1 + tz_2}(\alpha + \beta / z_1) - \frac{tz_2}{(1 - t)z_1 + tz_2}(\alpha + \beta / z_2) \\&= -\alpha - \frac{\beta}{(1 - t)z_1 + tz_2} \\&= -(\alpha + \beta / z)\end{aligned}$$

Prospettiva con Profondità

Da quest'ultima, per avere il valore originale z possiamo utilizzare la formula:

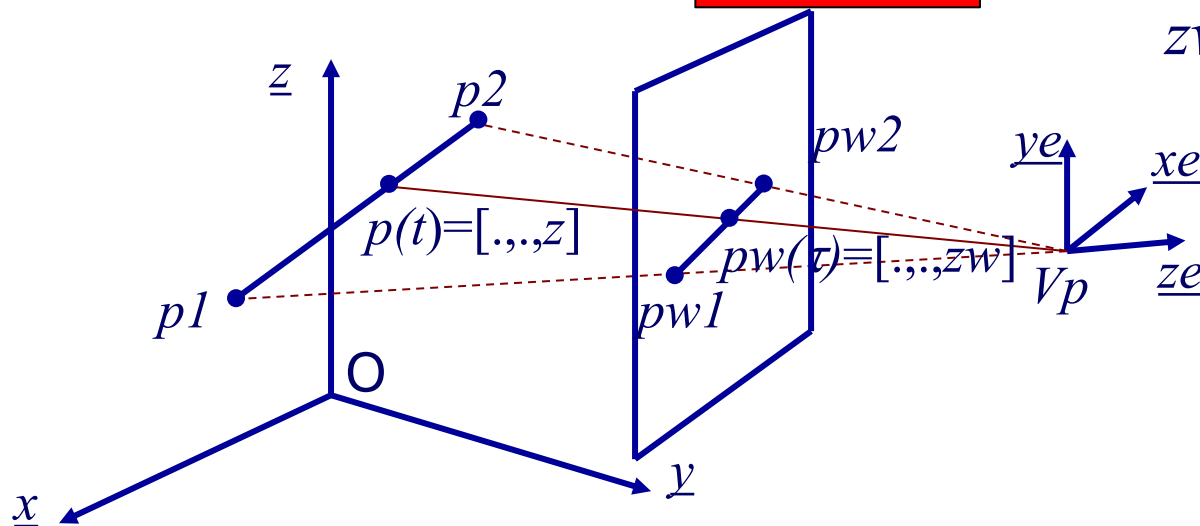
$$z = -\frac{\beta}{zw + \alpha}$$

dove:

$$zw = (1 - \tau) zw1 + \tau zw2$$

oppure, da τ :

$$t = \frac{\tau \cdot z_1}{\tau \cdot z_1 + (1 - \tau) \cdot z_2}$$



Si noti che possiamo usare le coordinate proiettate corrette e poi trasformate (window-viewport) xv e yv per il disegno e zw per la profondità.

Ma per il calcolo corretto di colore, coordinate texture, ecc.?



Prospettiva con Profondità

Verifichiamo per piani: se un triangolo viene trasformato in un triangolo dovrà esistere una relazione tra i parametri delle due forme parametriche.

Sia $p(u,v,s) = u p1 + v p2 + s p3$ $u,v,s \in [0, 1]$ $u+v+s=1$
un triangolo nello spazio dell'osservatore con $p_i = [x_i, y_i, z_i]$ $i=1,2,3$.
Siano $pw_i = [xw_i, yw_i, zw_i]$ $i=1,2,3$ i punti ottenuti applicando la trasformazione, cioè $pw_i = [-x_i/z_i, -y_i/z_i, -(\alpha+\beta/z_i)]$ ed il triangolo relativo sia

$pw(u',v',s') = u' pw1 + v' pw2 + s' pw3$ $u',v',s' \in [0, 1]$, $u'+v'+s'=1$.

Che relazione c'è tra u, v, s e u', v', s' ? La relazione si ricava imponendo che $p(u, v, s)$ venga trasformato in $pw(u', v', s')$; guardiamo l'ascissa del punto trasformato e a cosa corrisponde:

$$-\frac{ux_1 + vx_2 + sx_3}{uz_1 + vz_2 + sz_3} = -u' \frac{x_1}{z_1} - v' \frac{x_2}{z_2} - s' \frac{x_3}{z_3}$$



Prospettiva con Profondità

Si ricavano le relazioni dirette:

$$(6) \quad u' = \frac{u \cdot z_1}{u \cdot z_1 + v \cdot z_2 + s \cdot z_3} \quad (7) \quad v' = \frac{v \cdot z_2}{u \cdot z_1 + v \cdot z_2 + s \cdot z_3}$$

$$s' = \frac{s \cdot z_3}{u \cdot z_1 + v \cdot z_2 + s \cdot z_3} \quad (8)$$

Dalle (6), (7) e (8) si ricava:

$$\frac{u'}{z_1} + \frac{v'}{z_2} + \frac{s'}{z_3} = \frac{1}{u \cdot z_1 + v \cdot z_2 + s \cdot z_3} \quad (9)$$



Prospettiva con Profondità

... infatti ponendo:

$$fz = \frac{u'}{z_1} + \frac{v'}{z_2} + \frac{s'}{z_3} = \frac{1}{u \cdot z_1 + v \cdot z_2 + s \cdot z_3} \quad (9')$$

dalle (6), (7) e (8) si hanno le:

$$(6') \quad u = \frac{u'}{z_1 \cdot fz}$$

$$(7') \quad v = \frac{v'}{z_2 \cdot fz}$$

$$(8') \quad s = \frac{s'}{z_3 \cdot fz}$$



Prospettiva con Profondità

Verifichiamo che la proiezione (1) trasforma piani in piani. Si considera la proiezione di un punto $p(u,v,s)$ del piano per $p1-p2-p3$ e si verifica che stia sul piano per $pw1-pw2-pw3$.

$$pw = \left[-\frac{u \cdot x_1 + v \cdot x_2 + s \cdot x_3}{u \cdot z_1 + v \cdot z_2 + s \cdot z_3}, -\frac{u \cdot y_1 + v \cdot y_2 + s \cdot y_3}{u \cdot z_1 + v \cdot z_2 + s \cdot z_3}, -\left(\alpha + \frac{\beta}{u \cdot z_1 + v \cdot z_2 + s \cdot z_3}\right) \right]$$

dalle (6), (7) e (8) e dalla (9), si ha:

$$pw = \left[-u' \frac{x_1}{z_1} - v' \frac{x_2}{z_2} - s' \frac{x_3}{z_3}, -u' \frac{y_1}{z_1} - v' \frac{y_2}{z_2} - s' \frac{y_3}{z_3}, -u' \left(\alpha + \frac{\beta}{z_1} \right) - v' \left(\alpha + \frac{\beta}{z_2} \right) - s' \left(\alpha + \frac{\beta}{z_3} \right) \right]$$



Prospettiva con Profondità

Dalle relazioni (6), (7) e (8) si ricavano le relazioni inverse:

$$u' = \frac{u \cdot z_1}{u \cdot z_1 + v \cdot z_2 + s \cdot z_3} \quad v' = \frac{v \cdot z_2}{u \cdot z_1 + v \cdot z_2 + s \cdot z_3} \quad s' = \frac{s \cdot z_3}{u \cdot z_1 + v \cdot z_2 + s \cdot z_3}$$

Combinare con u' , v' , s' le coordinate proiettate, permette di avere i valori originali proiettati, in particolare vediamo la componente z :

$$zw = -u'(\alpha + \beta / z_1) - v'(\alpha + \beta / z_2) - s'(\alpha + \beta / z_3)$$

$$= -\frac{uz_1}{uz_1 + vz_2 + sz_3}(\alpha + \beta / z_1) - \frac{vz_2}{uz_1 + vz_2 + sz_3}(\alpha + \beta / z_2) - \frac{sz_3}{uz_1 + vz_2 + sz_3}(\alpha + \beta / z_3)$$

$$= -\alpha - \frac{\beta}{uz_1 + vz_2 + sz_3}$$

$$= -(\alpha + \beta / z)$$



Prospettiva con Profondità

Da quest'ultima, per avere il valore originale z possiamo utilizzare la formula:

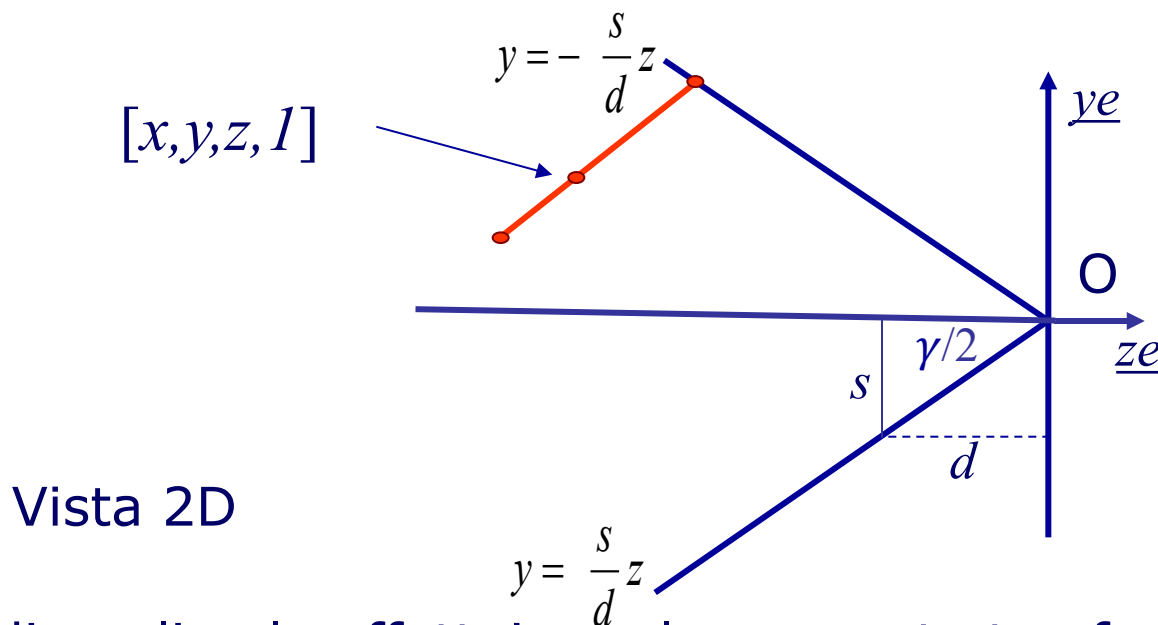
$$z = -\frac{\beta}{zw + \alpha} \quad (10)$$

Si noti che possiamo usare le coordinate proiettate corrette e poi trasformate (window-viewport) x_v e y_v per il disegno e zw per la profondità.

Ma per il calcolo corretto di colore, coordinate texture, ecc., come si può fare?

Prospettiva con Profondità

Cerchiamo di capire di più sulla proiezione (1):



prima di applicarla effettuiamo la seguente trasformazione di scala:

$$[xs, ys, zs, ws]^T = P [x, y, z, l]^T$$

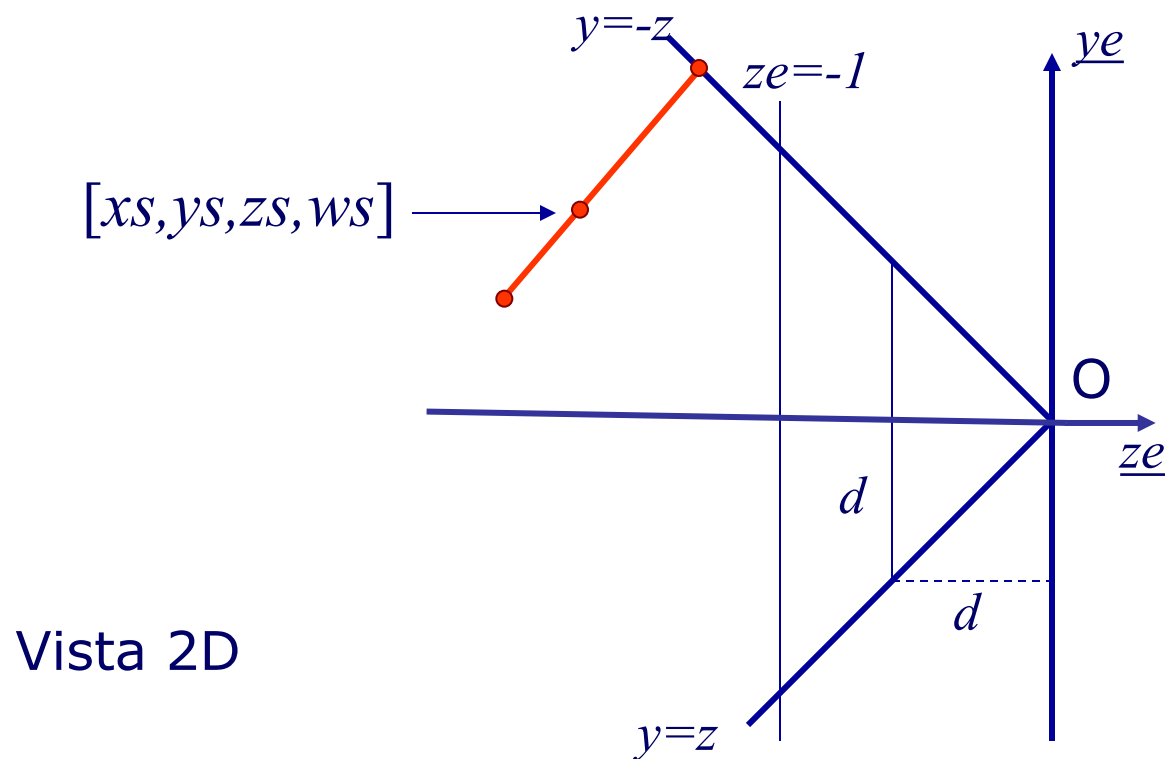
con

$$P = \begin{pmatrix} d/s \cdot \ar & 0 & 0 & 0 \\ 0 & d/s & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

ed $s = d \cdot \tan(\gamma/2)$

Prospettiva con Profondità

Otterremo così una piramide di vista di 90 gradi, od anche con le facce laterali che sono i piani bisettori, e senza che vengano alterate le coordinate z dei punti

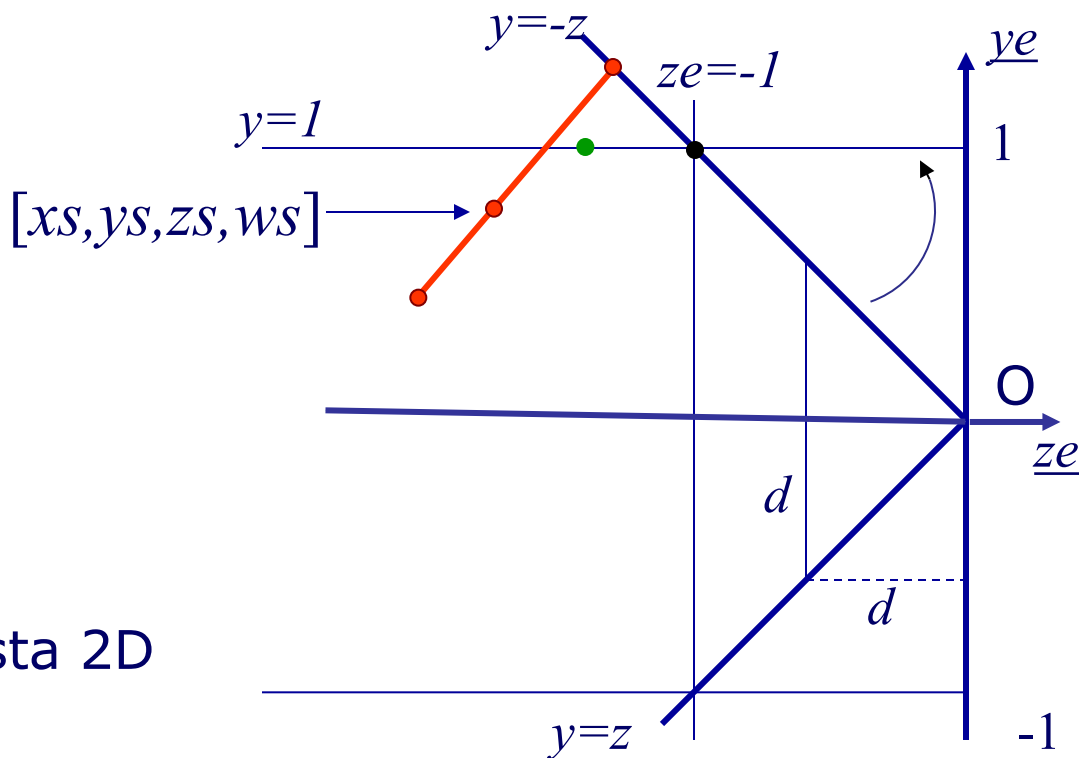


Prospettiva con Profondità

Applichiamo ora la proiezione (1):

i punti della forma $[\bullet, -z, z]$ vengono trasformati in $[\bullet, 1, -(\alpha + \beta/z)]$

La retta $y=-z$ viene trasformata nella retta $y=1$;



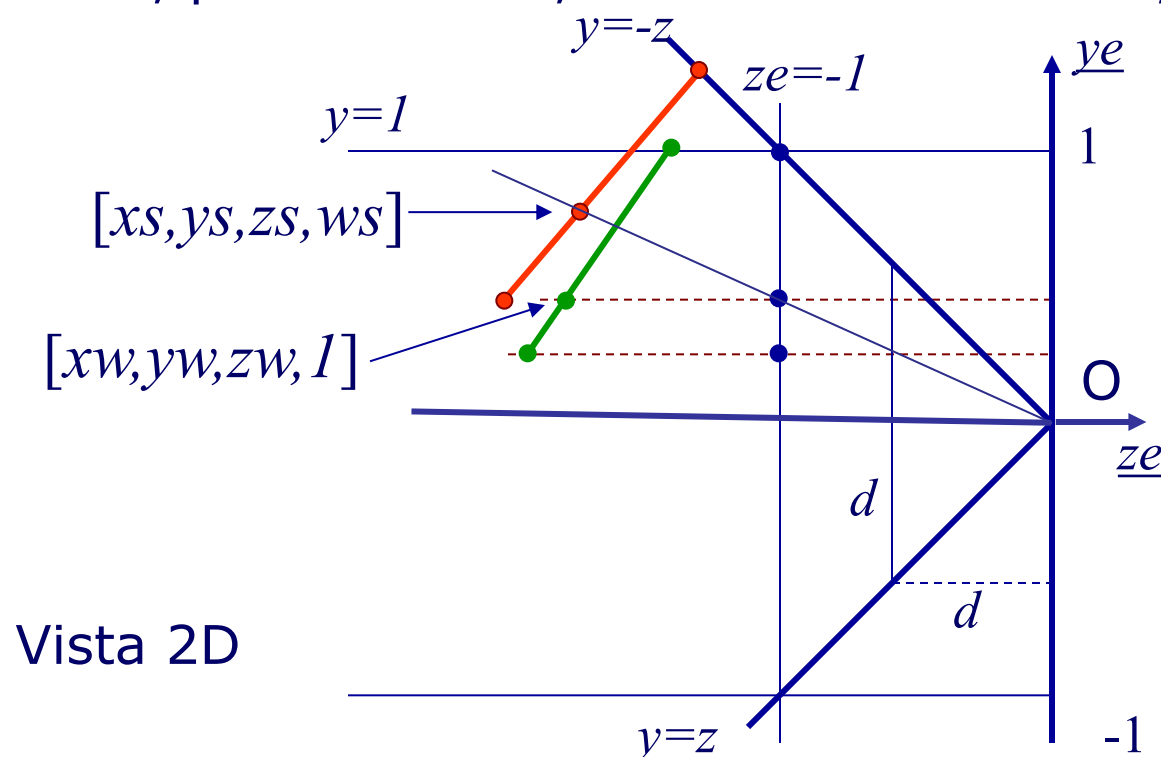
Vista 2D

Prospettiva con Profondità

tutte le rette $y=-mz$ uscenti dall'origine e di pendenza $-m$ saranno trasformate nelle rette $y=m$.

In pratica ruotano sul punto di intersezione di $y=-mz$ con $ze=-1$.

Ancora, punti allineati, una volta trasformati, saranno allineati.





Prospettiva con Profondità

E' poi possibile scegliere α e β in modo da ottenere un intervallo conveniente per i valori zw trasformati?

Scegliendo $\beta > 0$ si conserva la nozione intuitiva di ordinamento, cioè se un punto ha un valore z maggiore di un altro, il suo trasformato zw resterà maggiore (se $\beta < 0$ l'ordine sarà invertito).

Siano dati due punti tali che $z_1 < z_2$, e **verifichiamo** che $zw_1 < zw_2$ con $zw_i = -(\alpha + \beta / z_i)$ se $\beta > 0$.

Sarà: $\frac{1}{z_1} > \frac{1}{z_2}$; e se $\beta > 0$ $-(\alpha + \frac{\beta}{z_1}) < -(\alpha + \frac{\beta}{z_2})$ e $zw_1 < zw_2$ c.v.

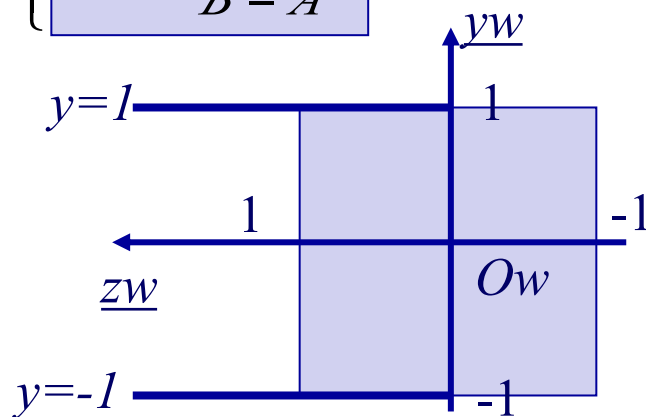
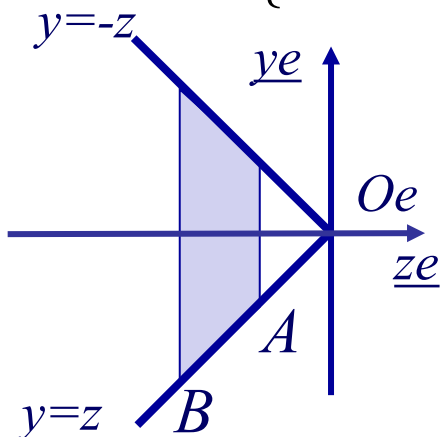
Prospettiva con Profondità

Determiniamo i valori α e β in modo che i valori zs nell'intervallo $[B, A]$ con $A, B < 0$ ($B < A$ perché negativi), siano trasformati in $zw \in [-1, 1]$, con A che va in -1 e B in 1 ; sarà:

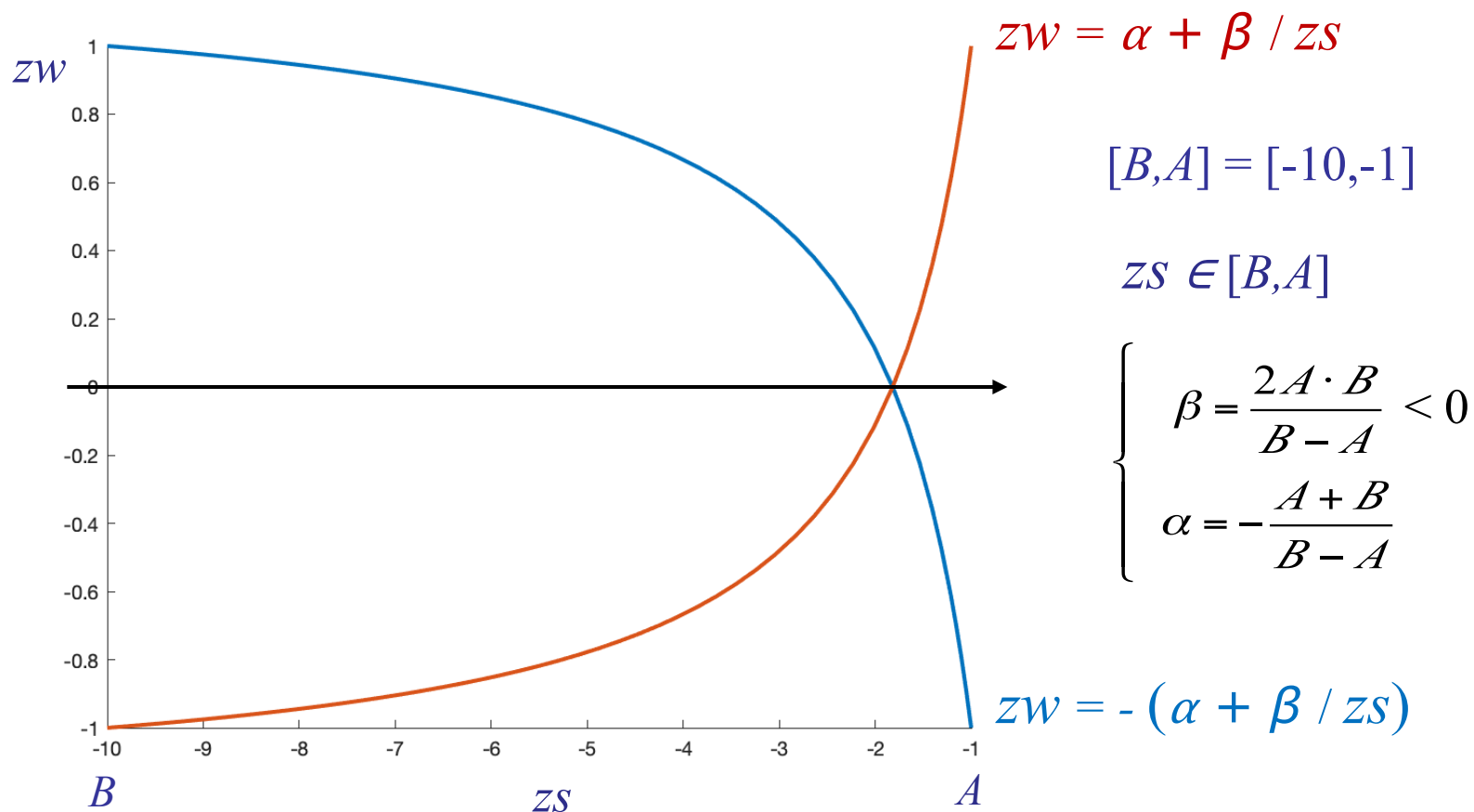
$$\begin{cases} -1 = -(\alpha + \beta / A) \\ 1 = -(\alpha + \beta / B) \end{cases} \quad \begin{cases} -A = -(\alpha A + \beta) \\ B = -(\alpha B + \beta) \end{cases} \quad \begin{cases} \beta = A(1 - \alpha) \\ B = -\alpha B - A(1 - \alpha) \end{cases}$$

$$\begin{cases} \beta = A(1 - \alpha) \\ \alpha = -\frac{A + B}{B - A} \end{cases}$$

$$\begin{cases} \beta = \frac{2A \cdot B}{B - A} \\ \alpha = -\frac{A + B}{B - A} \end{cases} \quad (11)$$



Immagini...amo

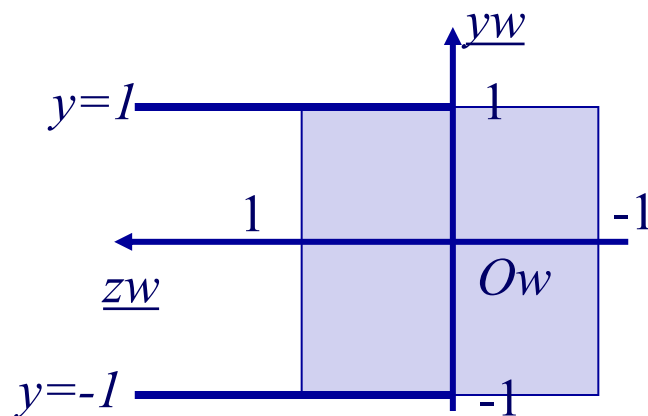
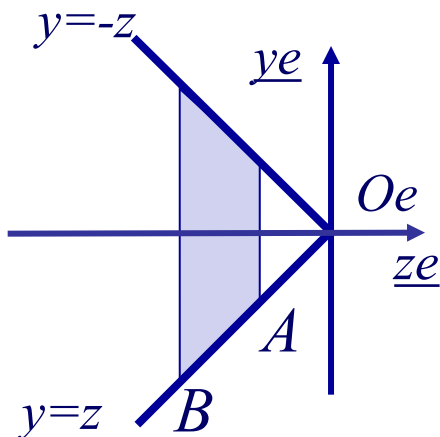


Prospettiva con Profondità

Il cubo WebGL è un sistema sinistrorso (si guarda nella direzione Z positiva). Allora se viene usata la trasformazione

$$zw = -(\alpha + \beta / zs)$$

la piramide di vista verrà trasformata nel cubo WebGL; l'ordinamento viene mantenuto sebbene venga invertita la relazione di profondità ($\beta < 0$), cioè i triangoli più lontani dall'osservatore hanno ora profondità (coordinata zw) maggiore.





Prospettiva con Profondità

Riassumendo, la proiezione in WebGL sarà:

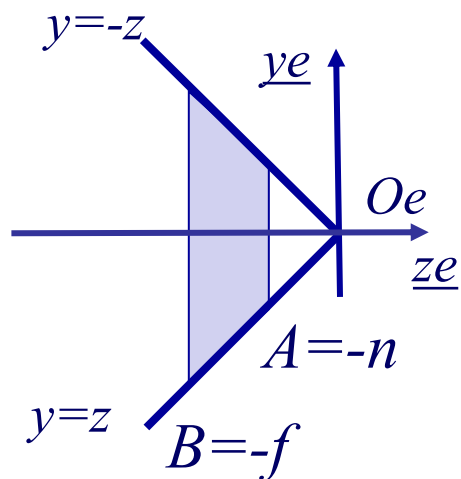
$$\left\{ \begin{array}{l} x_w = -x_s / z_s \\ y_w = -y_s / z_s \\ z_w = -(\alpha + \beta / z_s) \end{array} \right. \quad \left\{ \begin{array}{l} \beta = \frac{2A \cdot B}{B - A} \\ \alpha = -\frac{A + B}{B - A} \end{array} \right.$$

$$\begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} = \begin{bmatrix} -x_e d/s / z_e \\ -y_e d/s / z_e \\ -(\alpha + \beta / z_e) \end{bmatrix}$$

ricordiamo che $z_e < 0$, per cui si correggono i segni in $-x_e d/s / z_e$ e in $-y_e d/s / z_e$ così da avere le prime due coordinate positive.

Osservazione

Essendo A e B negativi, spesso si indicano con $-n$ (*near*) e $-f$ (*far*), con n ed f positivi (sono i parametri che si passano alla m4.perspective); in questo caso le formule per α e β saranno scritte come:



$$\left\{ \begin{array}{l} \beta = \frac{2A \cdot B}{B - A} = \frac{2 \cdot n \cdot f}{n - f} \\ \alpha = -\frac{A + B}{B - A} = \frac{f + n}{n - f} \end{array} \right.$$



Riassumendo

Scriviamo la trasformazione di vista in forma matriciale in coordinate omogenee:

$$[x_s, y_s, z_s, w_s]^T = P \cdot V \cdot [x, y, z, 1]^T$$

quindi:

$$[x_w, y_w, z_w, 1]^T = [x_s/w_s, y_s/w_s, z_s/w_s, 1]^T$$

dove V è la matrice di cambio sistema di riferimento e P è la matrice di proiezione:

$$P = \begin{pmatrix} d/s/ar & 0 & 0 & 0 \\ 0 & d/s & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

con

$$d/s = 1 / \tan (\gamma/2)$$

ar = aspect-ratio (Viewport)
= *width* / *height*

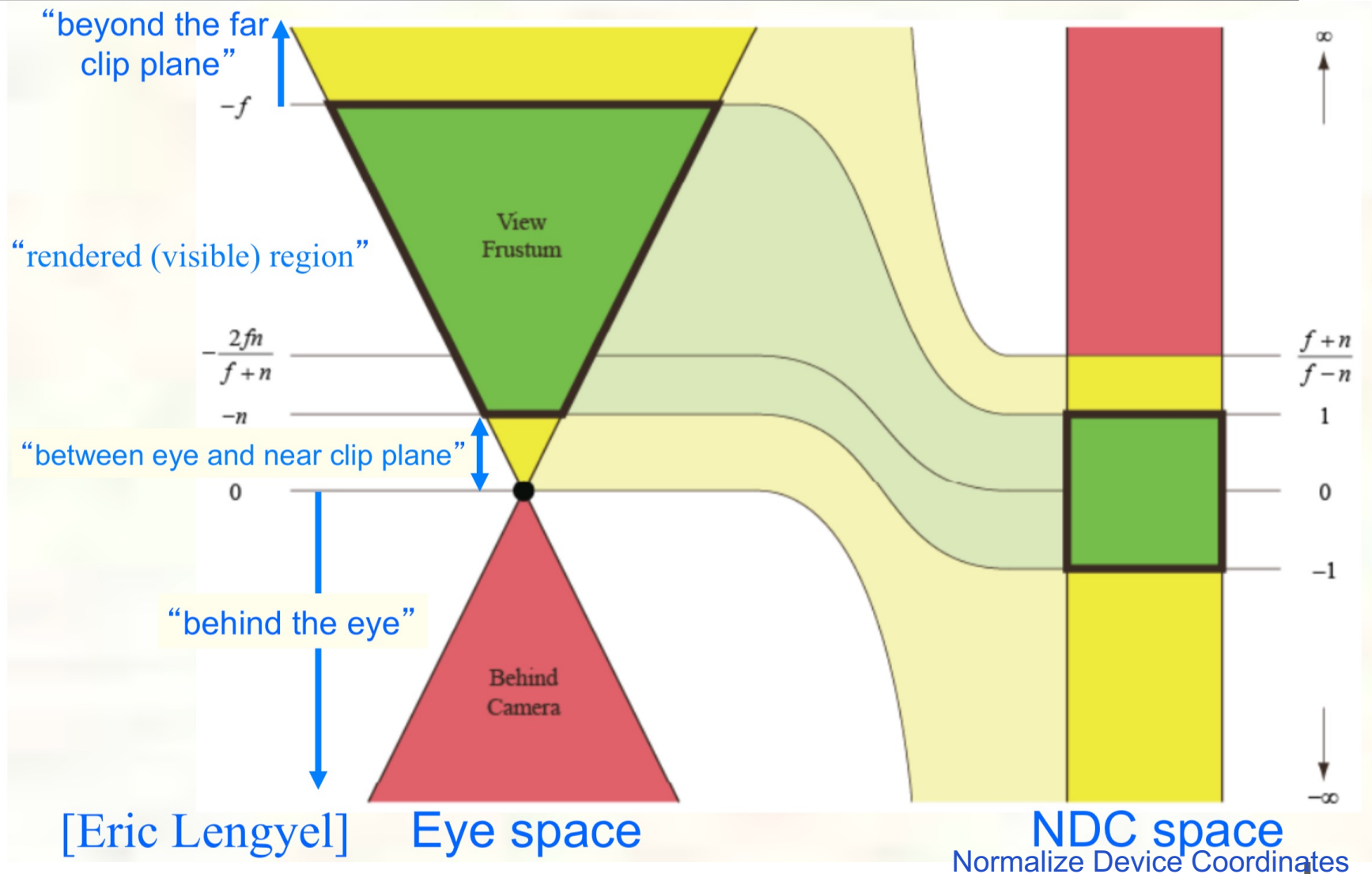
Riassumendo

$$\begin{bmatrix} xS \\ yS \\ zS \\ wS \end{bmatrix} = P \cdot V \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = P \cdot \begin{bmatrix} xe \\ ye \\ ze \\ 1 \end{bmatrix} = \begin{bmatrix} xe \, d/s/ar \\ ye \, d/s \\ \alpha ze + \beta \\ -ze \end{bmatrix}$$

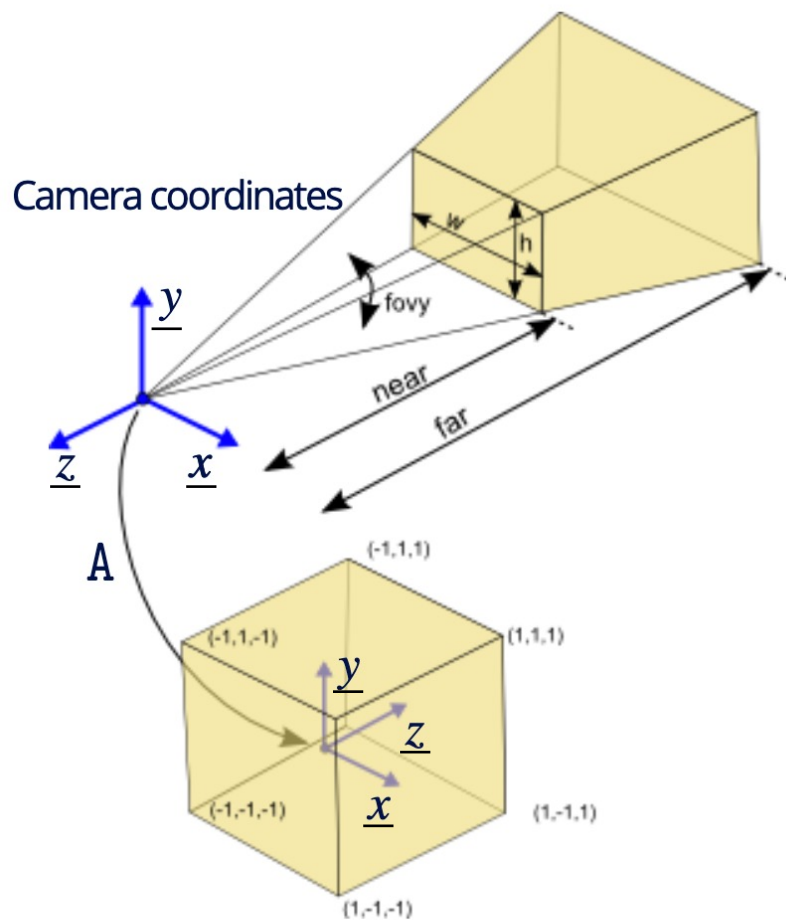
quindi, procedendo con la **divisione prospettica**:

$$\begin{bmatrix} xw \\ yw \\ zw \\ 1 \end{bmatrix} = \begin{bmatrix} xs/ws \\ ys/ws \\ zs/ws \\ ws/ws \end{bmatrix} = \begin{bmatrix} -xe \, d/s/ar / ze \\ -ye \, d/s / ze \\ -(\alpha + \beta / ze) \\ 1 \end{bmatrix}$$

Immagini...amo



Osservazione



Quando si usano le
 "normalize device
 coordinates" l'asse z viene
 rovesciato rispetto al
 sistema di coordinate della
 camera;
 si tratta quindi di un sistema
 di coordinate sinistrorso.

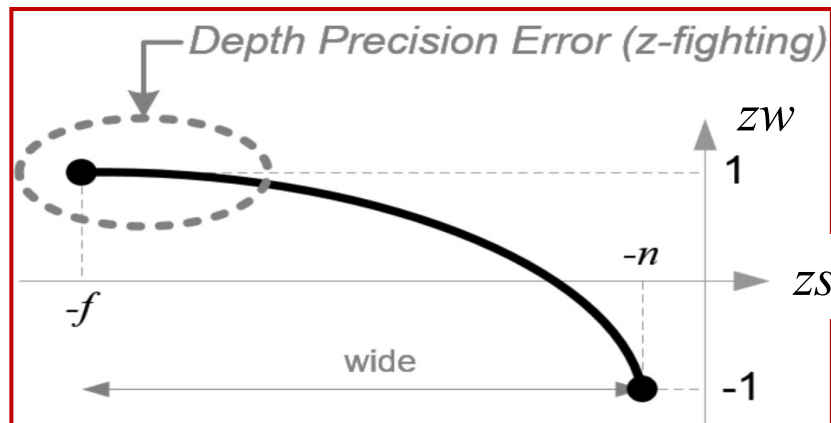
Osservazione

Prima di andare avanti, diamo ancora un'occhiata alla relazione

$$zw = -(\alpha + \beta / zs)$$

E' una funzione razionale, ma non lineare tra zw e zs .

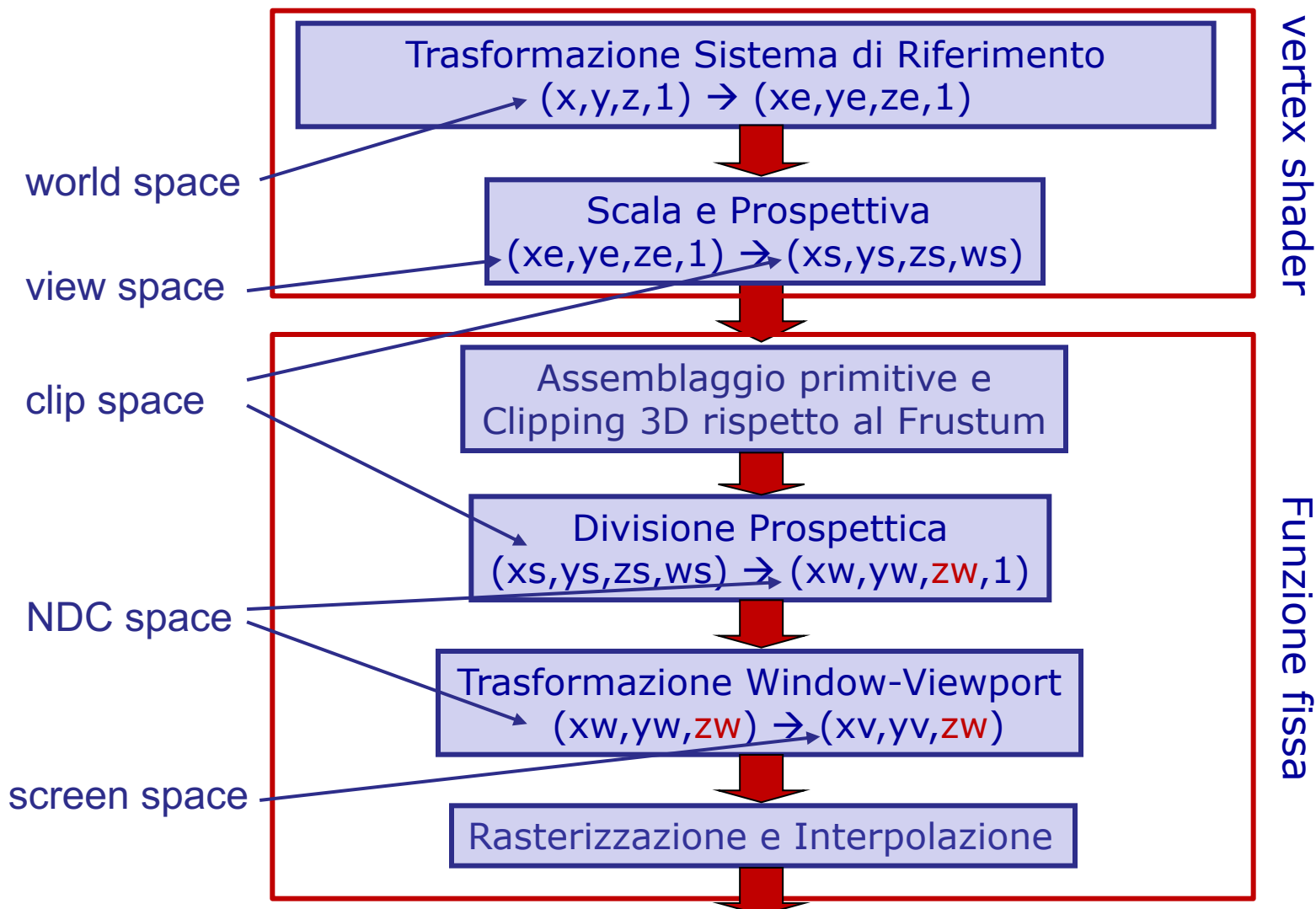
Il suo andamento comporta una buona precisione sul piano *near*, ma bassa precisione sul piano *far*. Se l'intervallo $[-f, -n]$ è grande si ha un problema di precisione sulla profondità (z-fighting); un piccolo cambiamento di zs vicino al piano *far* non cambia molto il valore di zw .



In conclusione, la distanza tra n ed f dovrebbe essere la più piccola possibile per ridurre al minimo il problema della precisione nel depth-buffer.



Pipeline di Rendering o di Vista





Clipping 3D

Il clipping 3D viene effettuato dalla parte fissa della pipeline, tra il vertex shader e il fragment shader, in clip-space (coordinate omogenee) dopo aver assemblato i vertici in linee o triangoli.

Clipping 3D rispetto ai sei 'piani' $x=\pm ws$, $y=\pm ws$, $z=\pm ws$

Viene effettuato prima della divisione prospettica utilizzando le coordinate $[xs, ys, zs, ws]^T = [..., ..., ..., -ze]^T$ infatti i punti, le linee e i triangoli devono stare all'interno del tronco di piramide di vista i cui piani laterali, frontale e retro in coordinate omogenee corrispondono alle sei equazioni:

$$x = \pm ws, y = \pm ws, z = \pm ws$$

A questo punto, WebGL effettua la **divisione prospettica**, tutto si trova in **Normalized Device Coordinates (NDC)** nel cubo WebGL e i vertici clippati e poi sottoposti a divisione prospettica sono tali che:

$$[xw, yw, zw, 1]^T \in [-1, 1]^3 = \text{cubo WebGL}$$



Rasterizzazione e Interpolazione

Dopo la trasformazione window-viewport si applica la rasterizzazione al “triangolo 2D” .

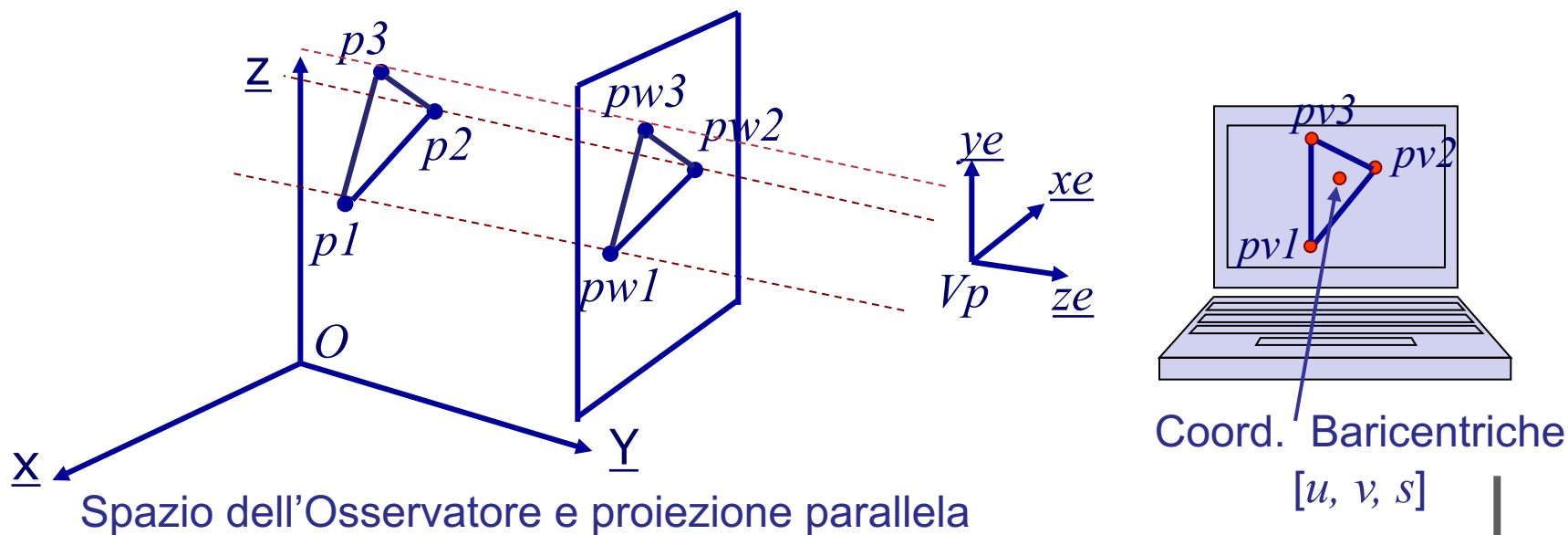
Dalle coordinate $[x_w, y_w, z_w]$ ottenute per proiezione con correzione prospettica dei vertici, si ottengono le coordinate $[x_v, y_v, z_v]$ dei vertici proiettati;

l'algoritmo di rasterizzazione genererà le coordinate $[x_v, y_v]$ dei pixel del triangolo 2D, e una coordinata z_v che rappresenta la profondità del punto 3D nello Spazio del Piano di Proiezione a cui il pixel corrisponde, infatti l'algoritmo di rasterizzazione, per ogni pixel interno al triangolo determinerà anche la sua profondità.

Rasterizzazione e Interpolazione

Siano $[u, v, s]$ le coordinate baricentriche del pixel $[x_v, y_v]$ che si sta considerando, allora la profondità z_v sarà determinata come:

$$z_v = u z_{w1} + v z_{w2} + s z_{w3}$$





Rasterizzazione e Interpolazione

Ripetiamo e andiamo oltre:

Ai tre vertici $[x_w, y_w, z_w]$ di ogni triangolo in Normalized Device Coordinates, sono associati eventuali attributi come colore $[r, g, b, \textit{alpha}]$, coordinate texture $[u, v]$, normali $[n_x, n_y, n_z]$.

Ai tre vertici viene applicata la trasformazione window-viewport ottenendo i vertici in coordinate schermo.

Grazie alle coordinate baricentriche si determinano i pixel $[x_v, y_v]$ del triangolo e lo scalare floating point z_v che rappresenterà la profondità del pixel e che sarà utile per lo Z-buffer.

Ma come si possono ottenere gli attributi dei vertici trasformati correttamente?



Rasterizzazione e Interpolazione

Indichiamo con a la coordinata di un attributo (per esempio r di $[r, g, b, \alpha]$), allora per ogni attributo di un vertice calcoliamo:

$$a' = a / w$$
$$w' = 1 / w$$

Interpolazione (lineare sul triangolo) utilizzando le coordinate baricentriche $[u, v, s]$

$$a'_{interp} = u a'_1 + v a'_2 + s a'_3$$
$$w'_{interp} = u w'_1 + v w'_2 + s w'_3$$

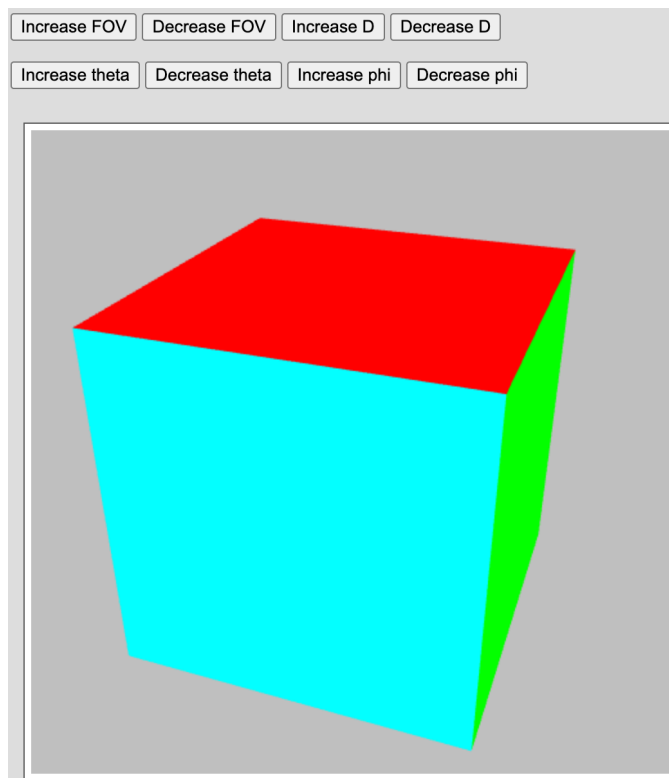
Recupero del valore corretto

$$a_{correct} = a'_{interp} / w'_{interp}$$

Questo produce un risultato coerente con la prospettiva.

Tutto questo avviene automaticamente nella pipeline fissa

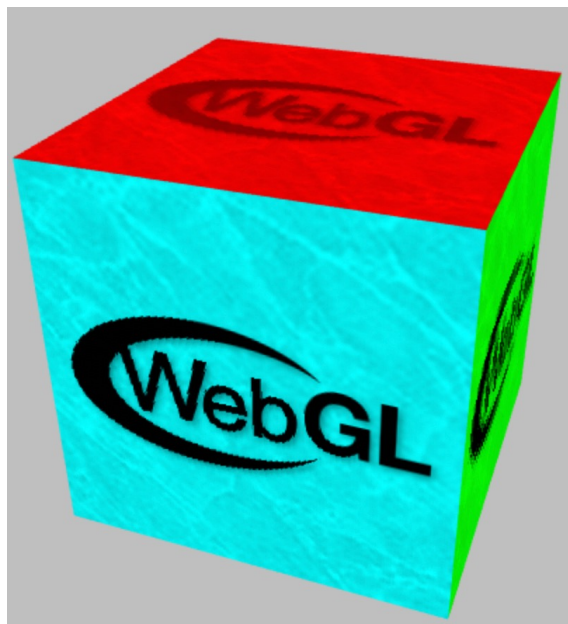
Codici di esempio



codice:

`persp_cube_color_interct_webgl.html`
`persp_cube_color2_interact_webgl.html`

Codice di esempio



codice:

`persp_cube_texture_interact_webgl.html`



Esempi

I codici WebGL che visualizzano un cubo a facce colorate ([cube_color_....html](#)), implementano la rotazione di un cubo interattivamente o in animazione.

Riprendere i singoli codici e modificarli per ottimizzarli.

interazione (mouse):

[cube_color_interact_webgl.html](#)

testo e interazione (mouse):

[cube_color_interact_and_text_webgl.html](#)

animazione:

[cube_color_anim_webgl.html](#)

interazione (mouse), ma in webgl2:

[cube_color_interact_webgl2.html](#)

utilizzo di webglutils

[cube_color_interact_webglutils0.html](#)

[cube_color_interact_webglutils1.html](#)



Esempi

Il seguente codice WebGL visualizza un cubo che ruota in animazione con una texture disegnata in contesto 2d.

Animazione e texture:

[cube_texture_cubemap.html + js](#)



Esercizio 4

Si propongono i seguenti punti da realizzare modificando il codice
persp_cube_texture_interact_webgl.js

- 1.modificare l'associazione vertici delle facce del cubo e coordinate u,v texture/immagine
- 2.utilizzare come texture una immagine personale e la si applichi alle facce del cubo (la risoluzione dell'immagine sia una potenza di 2)
- 3.texturare il cubo/mesh dando le coordinate u,v texture/immagine relative ai vertici delle facce per realizzare uno sky-box (si usi l'immagine sky-box.png); successivamente si visualizzi la scena muovendo la camera dentro il cubo
- 4.modificare il codice per sperimentare la proiezione prospettica senza correzione (vedi immagine iniziale di queste slide)

Il codice realizzato si chiami:
persp_cube_texture_interact_webgl_es4.html



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Giulio Casciola
Dip. di Matematica
giulio.casciola@unibo.it